
Calclatorz

Arithmetic Expression Evaluator in C++

Test Cases for Error_Handler Module

Version 1.2

Arithmetic Expression Evaluator in C++	Version: 1.2
Test Cases for Error_Handler	Date: 2026-05-06

Revision History

Date	Version	Description	Author
2026-04-28	1.0	Formatting completed and uniform	Isaiah
2026-05-04	1.1	All sections completed	Isaiah
2026-05-06	1.2	Final polishing	Isaiah

Arithmetic Expression Evaluator in C++	Version: 1.2
Test Cases for Error_Handler	Date: 2026-05-06

Table of Contents

1. Purpose	4
2. Invalid Expression	4
3. Invalid Unary Minus	5
4. Invalid Decimal Point	6
5. Invalid Completed Number	7
6. Invalid Character	8
7. Invalid Operator Existence	9
8. Invalid Operator Placement	10
9. Invalid Closing Parenthesis	11
10. Invalid Opening Parenthesis	12
11. Invalid Postfix Operand Count	13
12. Invalid Division by Zero	14

Arithmetic Expression Evaluator in C++	Version: 1.2
Test Cases for Error_Handler	Date: 2026-05-06

Test Cases

1. Purpose

This *Test Cases* specification document for the Arithmetic Expression Evaluator in C++ defines the test cases for the `error_handler` module.

2. Invalid Expression

2.1 Test case identifier

EH-Tokenizer1

2.2 Test item

This test case makes sure an invalid expression follows the path:

`input -> tokenizer -> error_handler -> output`

And the `error_handler` appropriately identifies the error.

2.3 Input specifications

User input: [enter key]

The tokenizer input: `tokenize("/n")`

The error_handler input: `validateExpression("/n")`

2.4 Output specifications

The program output: `"Empty expression: input expression is empty"`

2.5 Environmental needs

The program must be prepared to accept an expression input.

Arithmetic Expression Evaluator in C++	Version: 1.2
Test Cases for Error_Handler	Date: 2026-05-06

3. Invalid Unary Minus

3.1 Test case identifier

EH-Tokenizer2

3.2 Test item

This test case makes sure an invalid expression follows the path:

`input -> tokenizer -> error_handler -> output`

And the `error_handler` appropriately identifies the error.

3.3 Input specifications

User input: `3 + -*`

The tokenizer input: `tokenize("3 + -*/n")`

The error_handler input: `validateUnaryMinus("3 + -*/n", 5)`

3.4 Output specifications

The program output: `"Invalid number at position 6: expected digits after unary minus"`

3.5 Environmental needs

The program must be prepared to accept an expression input.

Arithmetic Expression Evaluator in C++	Version: 1.2
Test Cases for Error_Handler	Date: 2026-05-06

4. Invalid Decimal Point

4.1 Test case identifier

EH-Tokenizer3

4.2 Test item

This test case makes sure an invalid expression follows the path:

`input -> tokenizer -> error_handler -> output`

And the `error_handler` appropriately identifies the error.

4.3 Input specifications

User input: `7 - 99.99.99`

The tokenizer input: `tokenize("7 - 99.99.99/n")`

The error_handler input: `validateDecimalPoint("7-99.99.99/n", 9)`

4.4 Output specifications

The program output: `"Invalid number at position 10: multiple decimal points in number"`

4.5 Environmental needs

The program must be prepared to accept an expression input.

Arithmetic Expression Evaluator in C++	Version: 1.2
Test Cases for Error_Handler	Date: 2026-05-06

5. Invalid Completed Number

5.1 Test case identifier

EH-Tokenizer4

5.2 Test item

This test case makes sure an invalid expression follows the path:

`input -> tokenizer -> error_handler -> output`

And the `error_handler` appropriately identifies the error.

5.3 Input specifications

User input: `7 - .`

The tokenizer input: `tokenize("7 - ./n")`

The error_handler input: `validateCompletedNumber("7 - ./n", 4)`

5.4 Output specifications

The program output: `"Invalid number at position 5: standalone decimal point is not a valid number"`

5.5 Environmental needs

The program must be prepared to accept an expression input.

Arithmetic Expression Evaluator in C++	Version: 1.2
Test Cases for Error_Handler	Date: 2026-05-06

6. Invalid Character

6.1 Test case identifier

EH-Tokenizer5

6.2 Test item

This test case makes sure an invalid expression follows the path:

`input -> tokenizer -> error_handler -> output`

And the `error_handler` appropriately identifies the error.

6.3 Input specifications

User input: `280 / &`

The tokenizer input: `tokenize("280 / &/n")`

The error_handler input: `validateCharacter("280 / &/n", 3)`

6.4 Output specifications

The program output: `"Invalid character at position 7: unexpected character '&'"`

6.5 Environmental needs

The program must be prepared to accept an expression input.

Arithmetic Expression Evaluator in C++	Version: 1.2
Test Cases for Error_Handler	Date: 2026-05-06

7. Invalid Operator Existence

7.1 Test case identifier

EH-Parser1

7.2 Test item

This test case makes sure an invalid expression follows the path:

input -> tokenizer -> parser -> error_handler -> output

And the error_handler appropriately identifies the error.

7.3 Input specifications

User input: 40 8

The tokenizer input: `tokenize("40 8/n")`

The parser input: `toPostfix(infixTokens[Token{Number, "40", 0},
Token{Number, "8", 3}])`

The error_handler input: `validateOperatorExistence(infixTokens, 1)`

7.4 Output specifications

The program output: "Syntax error at position 4: tokens must be separated by operators"

7.5 Environmental needs

The program must be prepared to accept an expression input.

Arithmetic Expression Evaluator in C++	Version: 1.2
Test Cases for Error_Handler	Date: 2026-05-06

8. Invalid Operator Placement

8.1 Test case identifier

EH-Parser2

8.2 Test item

This test case makes sure an invalid expression follows the path:

input -> tokenizer -> parser -> error_handler -> output

And the error_handler appropriately identifies the error.

8.3 Input specifications

User input: 50505*/12121

The tokenizer input: `tokenize("50505*/12121/n")`

The parser input: `toPostfix(infixTokens[Token{Number, "50505", 0},
Token{Multiply, "*", 5},
Token{Divide, "/", 6},
Token{Number, "12121", 7}])`

The error_handler input: `validateOperatorPlacement(infixTokens, 2)`

8.4 Output specifications

The program output: "Syntax error at position 7: unexpected operator placement"

8.5 Environmental needs

The program must be prepared to accept an expression input.

Arithmetic Expression Evaluator in C++	Version: 1.2
Test Cases for Error_Handler	Date: 2026-05-06

9. Invalid Closing Parenthesis

9.1 Test case identifier

EH-Parser3

9.2 Test item

This test case makes sure an invalid expression follows the path:

input -> tokenizer -> parser -> error_handler -> output

And the error_handler appropriately identifies the error.

9.3 Input specifications

User input: 8)

The tokenizer input: tokenize("8)/n")

The parser input: toPostfix(infixTokens[Token{Number, "8", 0},
Token{RightParen, ")", 1}])

The error_handler input: validateClosingParenthesis(operators[], 1)

9.4 Output specifications

The program output: "Mismatched parentheses at position 2: closing parenthesis does not have a matching opening parenthesis"

9.5 Environmental needs

The program must be prepared to accept an expression input.

Arithmetic Expression Evaluator in C++	Version: 1.2
Test Cases for Error_Handler	Date: 2026-05-06

10. Invalid Opening Parenthesis

10.1 Test case identifier

EH-Parser4

10.2 Test item

This test case makes sure an invalid expression follows the path:

input -> tokenizer -> parser -> error_handler -> output

And the error_handler appropriately identifies the error.

10.3 Input specifications

User input: (8

The tokenizer input: tokenize("(8/n")

The parser input: toPostfix(infixTokens[Token{LeftParen, "(", 0},
Token{Number, "8", 1}])

The error_handler input: validateOpeningParenthesis(leftParenIndices[0])

10.4 Output specifications

The program output: "Mismatched parentheses at position 1: opening
parenthesis does not have a matching closing parenthesis"

10.5 Environmental needs

The program must be prepared to accept an expression input.

Arithmetic Expression Evaluator in C++	Version: 1.2
Test Cases for Error_Handler	Date: 2026-05-06

11. Invalid Postfix Operand Count

11.1 Test case identifier

EH-Evaluator1

11.2 Test item

This test case makes sure an invalid expression follows the path:

input -> tokenizer -> parser -> evaluator -> error_handler -> output

And the error_handler appropriately identifies the error.

11.3 Input specifications

User input: 5 + ()

The tokenizer input: tokenize("5 + ()/n")

The parser input: toPostfix(infixTokens[Token{Number, "5", 0},
Token{Plus, "+", 2},
Token{LeftParen, "(", 4},
Token{RightParen, ")", 6}])

The evaluator input: evaluatePostfix(postfixTokens[Token{Number, "5", 0},
Token{Plus, "+", 2}])

The error_handler input: validatePostfixOperandCount(1, Token{Plus, "+", 2})

11.4 Output specifications

The program output: "Syntax error at position 3: not enough operands for operator '+'"

11.5 Environmental needs

The program must be prepared to accept an expression input.

Arithmetic Expression Evaluator in C++	Version: 1.2
Test Cases for Error_Handler	Date: 2026-05-06

12. Invalid Division by Zero

12.1 Test case identifier

EH-Evaluator2

12.2 Test item

This test case makes sure an invalid expression follows the path:

input -> tokenizer -> parser -> evaluator -> error_handler -> output

And the error_handler appropriately identifies the error.

12.3 Input specifications

User input: 28 / 0

The tokenizer input: tokenize("28 / 0/n")

The parser input: toPostfix(infixTokens[Token{Number, "28", 0},
Token{Divide, "/", 3},
Token{Number, "0", 5}])

The evaluator input: evaluatePostfix(postfixTokens[Token{Number, "28", 0},
Token{Number, "0", 5},
Token{Divide, "/", 3}])

The error_handler input: validateDivisionByZero(0.0, 5)

12.4 Output specifications

The program output: "Division by zero at position 6: cannot divide by zero"

12.5 Environmental needs

The program must be prepared to accept an expression input.